

UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Ciencias Exactas e Ingenierías



MATERIA: Análisis de Algoritmos

Clave: IL355

Sección: D01

Calendario: 2026 A

SPRINT 2

Integrantes:

Nayeli Ximena Luna Gallegos
Axel Guillermo Martinez Martinez
Juan Carlos Mijares Ahuatzi

Códigos:

220442484
220450053
2211344872

SPRINT 2 — Mejora de Detección, Zonificación y Clasificación

2.1 Introducción

En este sprint se refinaron los módulos existentes y se añadieron nuevas capacidades: mejor detección YOLO, zonificación del espacio, conteo por zona, detección de patrones y clasificación del estado del tráfico por frame.

Capacidades nuevas respecto al Sprint 1:

- Ajuste de parámetros YOLO para mayor precisión y extracción de centros de detección.
- División del plano en zonas y conteo independiente por región.
- Detección de patrones, clasificación del estado del tráfico y análisis de variaciones entre frames.

2.2 Información del Sprint2.3 Actividades Realizadas

Actividad	Responsable	Descripción	Entregable
Mejorar detección YOLO	Juan	Ajustar parámetros (conf, iou) para mayor precisión	Config ajustada + comparación
Extraer coordenadas	Juan	Obtener centros (cx, cy) de los vehículos detectados	Script + salida JSON/console
Dividir espacio en zonas	Ximena	Separar el plano en regiones para análisis independiente	Diagrama de zonas + código
Contar vehículos por zona	Ximena	Calcular cuántos vehículos hay en cada región	Script + tabla de resultados
Detectar patrones de tráfico	Axel	Analizar métricas para identificar concentración y distribución	Tabla por zona/frame
Clasificar estado del tráfico	Axel	Determinar fluido/medio/congestionado por densidad por frame	Función + tabla por frame
Analizar cambios entre frames	Axel	Comparar métricas entre frames consecutivos	Tabla de variación + tendencias

SPRINT 2 — Resultados por Integrante

Juan Carlos Mijares Ahuatzi — Mejora YOLO y Extracción de Coordenadas

Actividad 1: Mejorar Detección de Vehículos

Confidence threshold: El modelo ahora es más exigente. Ya no va a marcar cualquier cosa que pudiera llegar a considerar un vehículo. Solo mostrará vehículos si está al menos **60% seguro**.

IoU: Si hay un camión grande y el modelo intenta dibujar dos o tres cajas delimitadoras encima, la regla del IoU al 50% será más estricta limpiando esos duplicados. Dejará solo la caja con el porcentaje de confianza más alto.

Clases filtradas: En las clases filtradas no hubo ningún cambio. Sigue mostrando las clases “classes=[2, 3, 5, 7]” correspondientes a **2:** Coche (Car), **3:** Motocicleta (Motorcycle), **5:** Autobús (Bus), **7:** Camión (Truck).

Comparación Antes / Después

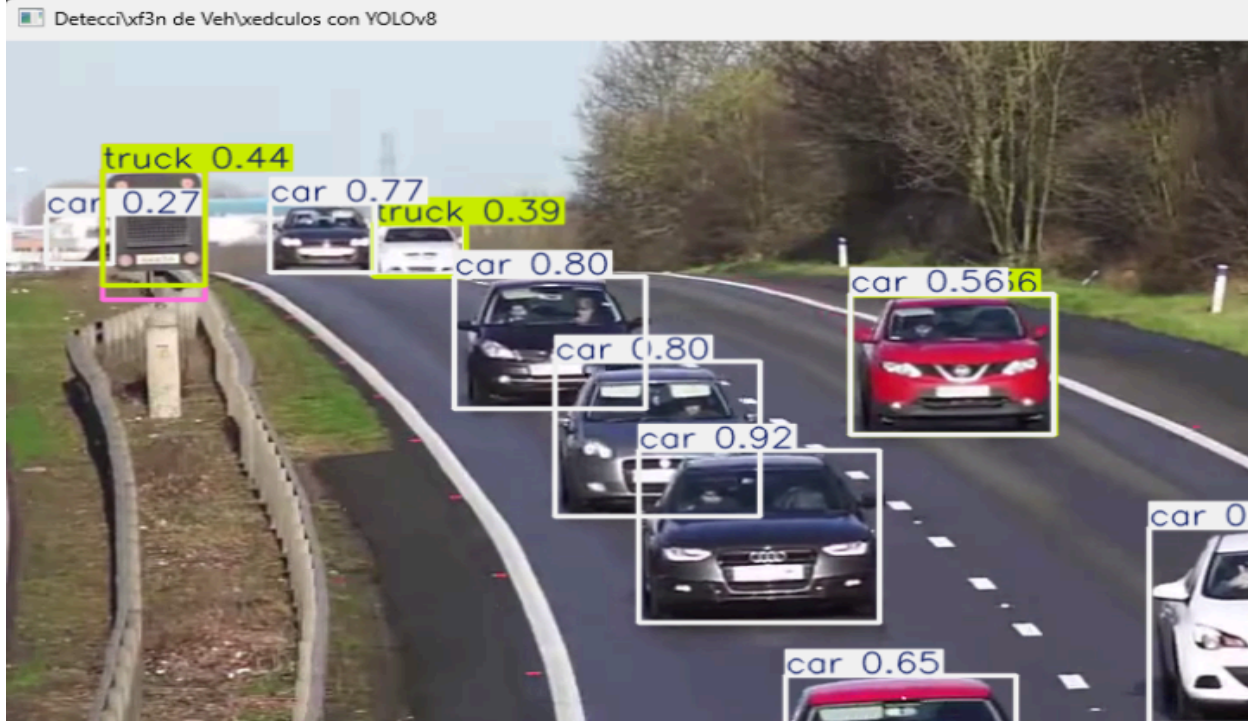
Métrica	Antes (Sprint 1)	Después (Sprint 2)
Falsos positivos	Altos (Más probables)	Bajos (Reducidos)
Conf. threshold	0.25 (Valor por defecto)	0.6 (Valor configurado)
IoU threshold	0.7 (Valor por defecto)	0.5 (Valor configurado)

Código de Configuración Ajustada

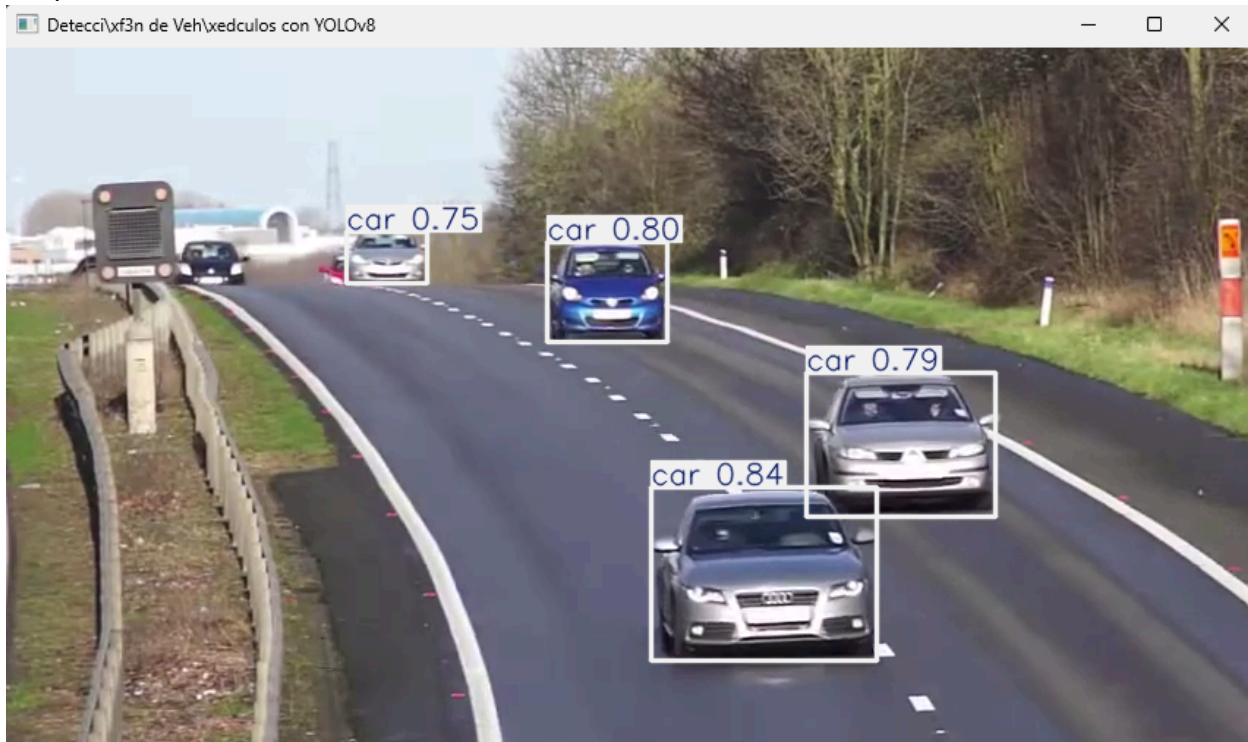
```
if exito:
    resultados = model(frame, classes=[2, 3, 5, 7], conf= 0.6, iou=0.5)
```

Evidencia Visual — Comparación

Antes:



Despu s:



Actividad 2: Extraer Coordenadas de Detección

Descripción breve de lo implementado:

La implementación de las coordenadas en este código permite localizar con exactitud espacial cada vehículo dentro del plano del video. Para lograrlo, el script utiliza la propiedad `xyxy` de YOLOv8, la cual extrae los valores numéricos correspondientes a la esquina superior izquierda y la esquina inferior derecha de la caja delimitadora de cada objeto. Dado que el modelo de inteligencia artificial devuelve estos datos como números decimales, el código los convierte inmediatamente a números enteros para que coincidan con la cuadrícula de píxeles reales del monitor. Finalmente, estas posiciones cartesianas se imprimen en la consola, creando un registro de texto en tiempo real sobre la ubicación exacta de cada detección frame por frame.

Código

```
for caja in cajas:
    # 1. Se extrae las coordenadas y las convierte a números enteros (píxeles)
    x1, y1, x2, y2 = caja.xyxy[0]
    x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)
```

Salida JSON / Consola

```
Posición -> CAR: Esquina Superior Izquierda (212, 142) | Esquina Inferior Derecha (256, 162)
Posición -> TRUCK: Esquina Superior Izquierda (311, 117) | Esquina Inferior Derecha (385, 174)
Posición -> TRUCK: Esquina Superior Izquierda (58, 89) | Esquina Inferior Derecha (117, 165)
Posición -> CAR: Esquina Superior Izquierda (115, 142) | Esquina Inferior Derecha (143, 161)
Posición -> CAR: Esquina Superior Izquierda (1, 121) | Esquina Inferior Derecha (63, 159)

0: 384x640 4 cars, 2 trucks, 31.4ms
Speed: 1.1ms preprocess, 31.4ms inference, 0.8ms postprocess per image at shape (1, 3, 384, 640)
Posición -> CAR: Esquina Superior Izquierda (269, 152) | Esquina Inferior Derecha (365, 230)
Posición -> CAR: Esquina Superior Izquierda (309, 117) | Esquina Inferior Derecha (385, 176)
Posición -> CAR: Esquina Superior Izquierda (213, 142) | Esquina Inferior Derecha (256, 162)
Posición -> TRUCK: Esquina Superior Izquierda (310, 117) | Esquina Inferior Derecha (385, 175)
Posición -> TRUCK: Esquina Superior Izquierda (58, 89) | Esquina Inferior Derecha (118, 167)
Posición -> CAR: Esquina Superior Izquierda (116, 143) | Esquina Inferior Derecha (143, 162)

0: 384x640 4 cars, 2 trucks, 32.3ms
Speed: 1.3ms preprocess, 32.3ms inference, 0.6ms postprocess per image at shape (1, 3, 384, 640)
Posición -> CAR: Esquina Superior Izquierda (270, 153) | Esquina Inferior Derecha (367, 233)
Posición -> CAR: Esquina Superior Izquierda (310, 117) | Esquina Inferior Derecha (387, 178)
Posición -> CAR: Esquina Superior Izquierda (212, 141) | Esquina Inferior Derecha (258, 162)
Posición -> TRUCK: Esquina Superior Izquierda (311, 117) | Esquina Inferior Derecha (387, 176)
Posición -> CAR: Esquina Superior Izquierda (115, 142) | Esquina Inferior Derecha (145, 162)
Posición -> TRUCK: Esquina Superior Izquierda (58, 90) | Esquina Inferior Derecha (117, 167)

0: 384x640 9 cars, 2 trucks, 31.3ms
Speed: 1.3ms preprocess, 31.3ms inference, 0.7ms postprocess per image at shape (1, 3, 384, 640)
Posición -> CAR: Esquina Superior Izquierda (272, 155) | Esquina Inferior Derecha (370, 234)
Posición -> CAR: Esquina Superior Izquierda (315, 117) | Esquina Inferior Derecha (389, 180)
Posición -> CAR: Esquina Superior Izquierda (213, 141) | Esquina Inferior Derecha (258, 163)
Posición -> TRUCK: Esquina Superior Izquierda (58, 89) | Esquina Inferior Derecha (118, 168)
```

Conclusión de Juan

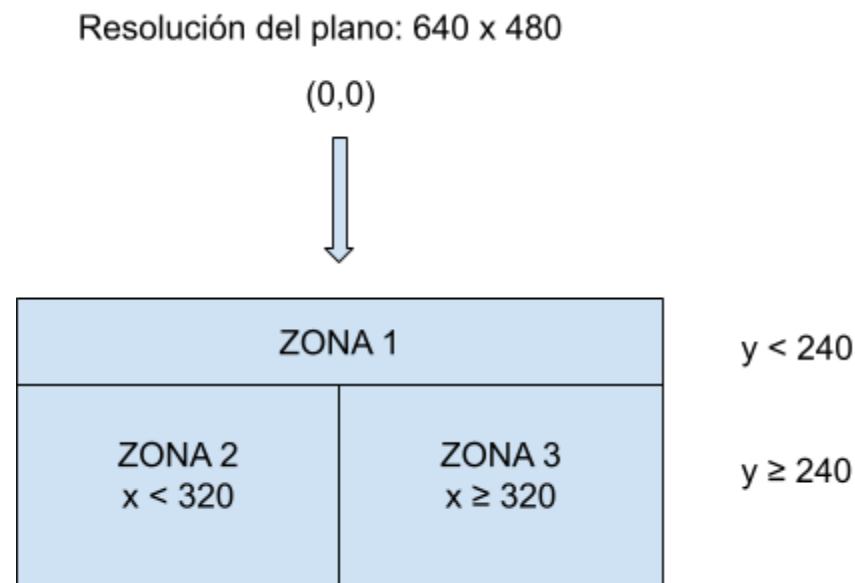
En conclusión, se modificó el script básico de detección de objetos en una herramienta optimizada para el análisis de tráfico vehicular mediante la aplicación práctica de conceptos de visión artificial: filtramos clases específicas (coches, motos, autobuses y camiones), ajustamos el umbral de confianza al 60% para evitar detecciones erróneas y afinamos el IoU al 50% para limpiar las cajas superpuestas. Adicionalmente, se implementó un sistema para localizar con exactitud espacial cada vehículo dentro del plano del video; esto se logró extrayendo las coordenadas (xyxy) de las cajas delimitadoras y convirtiendo sus valores decimales a números enteros (píxeles) para crear un registro en consola en tiempo real. Todo esto asegurando además un cierre limpio del programa con la gestión de memoria de OpenCV. El principal reto de este ejercicio fue lograr aterrizar la teoría abstracta en código real, descubriendo cómo YOLOv8 maneja estos parámetros "invisibles" por defecto y aprendiendo a ajustarlos de manera segura dentro del ciclo de lectura del video. Como próximos pasos, se espera evolucionar este script integrando un algoritmo de seguimiento (tracking) que asigne un ID único a cada vehículo detectado, lo que permitiría crear una línea virtual en el video para contar exactamente cuántos coches pasan por la calle o exportar las coordenadas registradas a un archivo para analizarlas después.

Nayeli Ximena Luna Gallegos — Zonificación y Conteo por Zona

Actividad 1: Dividir el Espacio en Zonas

En esta actividad se definieron 3 zonas dentro del plano de la imagen. La segmentación se realizó utilizando regiones rectangulares, donde se dividió el espacio en una franja superior (zona 1) y una franja inferior, la cual se subdivide en dos partes: izquierda (zona 2) y derecha (zona 3). En el código, estas zonas se representan mediante condiciones sobre las coordenadas (x, y), evaluando la posición de cada vehículo respecto a la mitad del ancho y alto del plano.

Diagrama de Zonas



Código de Segmentación

```
# CONFIGURACIÓN DEL PLANO
ANCHO = 640
ALTO = 480

def asignar_zona(x, y):
    # Zona 1 (arriba)
    if y < ALTO / 2:
        return "zona_1"

    # Zona 2 (abajo izquierda)
    elif x < ANCHO / 2:
        return "zona_2"

    # Zona 3 (abajo derecha)
    else:
        return "zona_3"

def segmentar_frame(vehiculos):
    resultado = []

    for v in vehiculos:
        zona = asignar_zona(v["x"], v["y"])

        vehiculo_con_zona = {
            "id": v["id"],
            "x": v["x"],
            "y": v["y"],
            "zona": zona
        }

        resultado.append(vehiculo_con_zona)

    return resultado

def segmentar_dataset(dataset):
    dataset_zonas = {}

    for frame, vehiculos in dataset.items():
        dataset_zonas[frame] = segmentar_frame(vehiculos)

    return dataset_zonas
```

Actividad 2: Contar Vehículos por Zona

En esta actividad se implementó un algoritmo que permite contar la cantidad de vehículos en cada zona definida dentro de cada frame. A partir de los datos segmentados, se recorren los vehículos y se incrementa un contador según la zona a la que pertenecen. Como resultado, se obtiene un resumen por frame que indica cuántos vehículos hay en cada región.

Código

```
def contar_por_zona(dataset_zonas):
    resultados = {}

    for frame, vehiculos in dataset_zonas.items():
        conteo = {
            "zona_1": 0,
            "zona_2": 0,
            "zona_3": 0
        }

        for v in vehiculos:
            zona = v["zona"]
            conteo[zona] += 1

        resultados[frame] = conteo

    return resultados
```

Tabla de Resultados

FRAME	ZONA 1	ZONA 2	ZONA 3
frame_0	1	2	2
frame_1	5	0	0
frame_2	2	1	2

Conclusión de Ximena

En este segundo sprint se avanzó de trabajar con datos simulados básicos a implementar una lógica más estructurada, ya que ahora los vehículos no solo se generan, sino que también se clasifican por zonas y se contabilizan dentro de cada una. Esto representa

una mejora respecto al Sprint 1, donde únicamente se organizaban los datos por frames sin analizarlos.

Uno de los principales retos fue entender cómo dividir correctamente el plano y cómo interpretar las coordenadas (x, y) para asignar cada vehículo a una zona adecuada, así como manejar la estructura de datos para obtener los conteos.

Como siguientes pasos, se propone integrar esta lógica con datos reales obtenidos del video mediante el modelo de detección, así como mejorar el análisis agregando métricas más completas del flujo vehicular.

Axel Guillermo Martinez Martinez — Patrones, Clasificación y Análisis Temporal

Actividad 1: Detectar Patrones de Tráfico

En esta actividad se implementó un método para identificar patrones de tráfico a partir de la distribución de vehículos en diferentes zonas (carriles).

Para ello, se analizaron las siguientes métricas:

Concentración de vehículos por zona, para detectar en qué carril existe mayor carga.

Distribución desigual, identificando si el tráfico está balanceado o concentrado en una sola zona.

Zonas de mayor actividad, determinando cuál carril domina en cada frame.

A partir de estas métricas, se estableció una clasificación simple de patrones, donde se identifica si un carril es dominante o si el tráfico está distribuido de manera uniforme.

Código

Python

```
def detectar_patron(zonas):
    total = sum(zonas.values())
    if total == 0:
        return "Sin datos"

    z1, z2, z3 = zonas["Z1"], zonas["Z2"], zonas["Z3"]

    p1 = z1 / total
    p2 = z2 / total
    p3 = z3 / total

    if max(p1, p2, p3) > 0.6:
        if p1 == max(p1, p2, p3):
            return "Alta concentración en carril 1"
        elif p2 == max(p1, p2, p3):
```

```

        return "Alta concentración en carril 2"
    else:
        return "Alta concentración en carril 3"

    if max(p1, p2, p3) - min(p1, p2, p3) > 0.3:
        return "Distribución desigual"

    return "Flujo equilibrado"

```

Tabla de Métricas por Zona/Frame

```

=== MÉTRICAS COMPLETAS ===

```

Frame	Veh	Dens	Z1	Z2	Z3	Disp	Estado	Patrón	Cambio
1	6	Media	3	0	3	311174.33	Moderado fluido	Distribución desigual	Sin referencia
2	2	Baja	0	1	1	31501.00	Fluido	Distribución desigual	Caída brusca
3	16	Alta	5	6	5	658373.88	Congestionado (disperso)	Flujo equilibrado	Incremento brusco

```

=== RENDIMIENTO ===

Vehículos    Tiempo (ms)
-----
100           0.7377
500           21.1938
1000          86.0335
2000         338.5085

```

Actividad 2: Clasificar Estado del Tráfico

En esta actividad se implementó una función para clasificar el estado del tráfico en cada frame, utilizando como base la densidad de vehículos previamente calculada.

Se definieron las siguientes reglas:

Densidad baja → Tráfico fluido

Densidad media → Tráfico moderado

Densidad alta → Tráfico congestionado

Esta clasificación permite interpretar el estado del tráfico de forma más clara, facilitando su análisis.

Código — Función de Clasificación

Python

```
def clasificar_trafico(densidad, dispersion):
```

```
if densidad == "Alta":
    if dispersion < 50000:
        return "Congestionado (compacto)"
    else:
        return "Congestionado (disperso)"

elif densidad == "Media":
    if dispersion < 40000:
        return "Moderado con acumulación"
    else:
        return "Moderado fluido"

else:
    return "Fluido"
```

Actividad 3: Analizar Cambios entre Frames

Se implementó un análisis de variación entre frames consecutivos, comparando el número de vehículos para identificar cambios en el tráfico.

Código

Python

```
def comparar_frames(actual, anterior):
    if not anterior:
        return "Sin referencia"

    diff = actual["n_vehiculos"] - anterior["n_vehiculos"]

    ratio = diff / (anterior["n_vehiculos"] + 1)

    if ratio > 0.5:
        return "Incremento brusco"
    elif ratio > 0.1:
        return "Incremento gradual"
    elif ratio < -0.5:
        return "Caída brusca"
    elif ratio < -0.1:
        return "Disminución gradual"
    else:
        return "Estable"
```

Conclusión de Axel

En este sprint se logró una mejora importante respecto al Sprint 1, ya que el sistema pasó de únicamente generar métricas básicas a interpretar el comportamiento del tráfico.

En el Sprint 1, el enfoque estaba en el procesamiento de datos por frame, obteniendo valores como número de vehículos, densidad y distribución por zonas. Sin embargo, estos datos no eran interpretados.

En el Sprint 2, se añadió la identificación de patrones de tráfico mediante el análisis de la distribución de vehículos por zona, clasificación automática del estado del tráfico (fluido, moderado o congestionado) y el análisis temporal mediante la comparación entre frames consecutivos para detectar cambios en el tráfico

Estas mejoras permiten que el sistema no solo mida el tráfico, sino que también lo interprete, lo que representa un avance significativo hacia un sistema más completo.

Como próximos pasos, se propone integrar estos análisis con datos reales provenientes del módulo de detección en video, así como ampliar el análisis mediante técnicas más avanzadas como agrupaciones (clustering) y relaciones entre vehículos.

SPRINT 2 — Conclusión General

Estado General del Proyecto

Actualmente, el sistema ha evolucionado desde una etapa inicial basada en datos simulados hacia una estructura más completa que integra detección, organización y análisis del tráfico. En este punto, el módulo de detección permite identificar vehículos en video y obtener sus coordenadas por frame, mientras que el módulo de zonas organiza estos datos dividiendo el espacio en regiones para analizar su distribución. Por su parte, el módulo de análisis procesa esta información para generar métricas, clasificar el estado del tráfico, identificar patrones y analizar cambios entre frames, lo que permite interpretar el comportamiento del tráfico y no solo medirlo. Sin embargo, el sistema aún trabaja en pruebas controladas y no en un flujo completamente continuo en tiempo real. Como siguiente paso, en el Sprint 3 se buscará implementar el procesamiento en tiempo real sin depender de archivos pregrabados, generar datos dinámicos por frame para conectar directamente los módulos, visualizar las zonas sobre el video en vivo y aplicar análisis espacial más avanzado mediante grafos y clustering, con el objetivo de integrar todo en un sistema completo y funcional.